

Spring Security & Logging

Spring ofrece una implementación del patrón ACL (Access Control List) para resolver necesidades de autenticación. Este patrón utiliza 3 conceptos fundamentales:

- Roles
- Recursos
- Permisos que aplican a un rol para un recurso

Spring Security surge como un proyecto Open Source de la comunidad, llamado *Acegi Security* que es adoptado en forma oficial por Spring. Si bien todos los major version numbers acompañan a Spring Framework, siendo un proyecto independiente, no hacen releases simultáneos y por tanto no mantiene siempre la misma versión.

Por tanto, debemos agregar una nueva dependencia al pom padre:

```
<spring-security.version>5.3.13.RELEASE</spring-security.version>
```

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-core</artifactId>
  <version>${spring-security.version}</version>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-web</artifactId>
  <version>${spring-security.version}</version>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-config</artifactId>
  <version>${spring-security.version}</version>
</dependency>
```

Luego, en el pom de webapp, ya que es en esta capa que vamos a validar el control de acceso, declaramos las mismas dependencias:

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-core</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-config</artifactId>
</dependency>
```

Ejecutamos `mvn eclipse:eclipse`, refrescamos el proyecto, y procedemos a configurar Spring Security. Por orden, vamos a mantener la configuración de Spring Security separada del resto de la aplicación, por lo que vamos a generar en el paquete

ar.edu.itba.paw.webapp.config una clase llamada WebAuthConfig. Dado que la misma será un objeto @Configuration procedemos a indicar en el web.xml que el contexto debe tener en cuenta esta configuración también al inicializarse, modificando el context-param para incluir una referencia al mismo:

```
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>
    ar.edu.itba.paw.webapp.config.WebConfig,
    ar.edu.itba.paw.webapp.config.WebAuthConfig,
  </param-value>
</context-param>
```

Y luego escribimos nuestra clase:

```
@Configuration
@EnableWebSecurity
@ComponentScan("ar.edu.itba.paw.webapp.auth")
public class WebAuthConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    private PawUserDetailsService userDetailsService;

    @Override
    protected void configure(final HttpSecurity http) throws Exception {
        http.userDetailsService(userDetailsService)
            .sessionManagement()
                .invalidSessionUrl("/login")
            .and().authorizeRequests()
                .antMatchers("/login").anonymous()
                .antMatchers("/admin/**").hasRole("ADMIN")
                .antMatchers("/**").authenticated()
            .and().formLogin()
                .usernameParameter("j_username")
                .passwordParameter("j_password")
                .defaultSuccessUrl("/", false)
                .loginPage("/login")
            .and().rememberMe()
                .rememberMeParameter("j_rememberme")
                .userDetailsService(userDetailsService)
                .key("mysupersecretketthatnobodyknowsabout")
                .tokenValiditySeconds((int) TimeUnit.DAYS.toSeconds(30))
            .and().logout()
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login")
            .and().exceptionHandling()
                .accessDeniedPage("/403")
            .and().csrf().disable();
    }

    @Override
    public void configure(final WebSecurity web) throws Exception {
```

```

        web.ignoring()
            .antMatchers("/css/**", "/js/**", "/img/**", "/favicon.ico", "/403");
    }
}

```

Como vimos en clase, es importante el orden de las reglas escritas para `authorizeRequests()`, ya que la primera que coincida con la url solicitada será aplicada y no se evaluarán las subsiguientes. Es por esto que la regla default `antMatchers("/**").authenticated()` es la última en aplicarse.

Por el uso de `csrf` de momento vamos a dejarlo desactivado, volveremos a hablar de esto cuando conversemos sobre seguridad de aplicaciones web.

Finalmente, configuramos el filtro que Spring Security utilizará para interceptar todas las solicitudes entrantes, agregando al final del `web.xml` lo siguiente:

```

<filter>
    <filter-name>springSecurityFilterChain</filter-name>
    <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-
class>
</filter>

<filter-mapping>
    <filter-name>springSecurityFilterChain</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

```

Como conversamos en clase, Spring Security configura automáticamente múltiples providers para distintos mecanismos de autenticación según la configuración que realicemos (user y password, cookie de rememberme, OAuth, Http-Authentication, etc.), pudiendo siempre el desarrollador crear uno propio.

Todos estos providers terminan obteniendo un `username`, con el que delegar al `UserDetailsService` para obtener el `User` y luego, de ser necesario, delega en un `PasswordEncoder` para hashear la password y compararla con la recuperada por el `UserDetailsService`. El `PasswordEncoder` utilizado por defecto es `PlainTextPasswordEncoder` que para nuestro ejemplo básico es suficiente. Igualmente, en una aplicación real (como la desarrollada durante el TP especial), debieramos configurar un buen `PasswordEncoder` como ser `BCryptPasswordEncoder`, y asegurarnos de usarlo en nuestra aplicación al momento de crear / modificar la password de nuestros usuarios.

Escribimos así nuestra implementación de `UserDetailsService`, el `PawUserDetailsService`:

```

@Component
public class PawUserDetailsService implements UserDetailsService {

    @Autowired
    private UserService us;

    @Override
    public UserDetails loadUserByUsername(final String username)
        throws UsernameNotFoundException {

```

```

    final User user = us.findByUsername(username);

    if (user == null) {
        throw new UsernameNotFoundException("No user by the name " +
username);
    }

    final Collection<? extends GrantedAuthority> authorities = Arrays.asList(
        new SimpleGrantedAuthority("ROLE_USER"),
        new SimpleGrantedAuthority("ROLE_ADMIN")
    );
    return new org.springframework.security.core.userdetails.User(username,
user.getPassword(), authorities);
}
}

```

Como vimos en clase, aunque no lo estamos utilizando, los múltiples constructores de la clase `User` de Spring Security permiten diferencia usuarios bloqueados, desactivados, con passwords expiradas, etc. y disparar los flujos correspondientes. Adicionalmente, estamos estableciendo por defecto que todos los usuarios tienen los roles `ROLE_ADMIN` y `ROLE_USER`, aunque en la práctica esto será generado dinámicamente basado en información de nuestro sistema.

Cabe destacar que las `authorities` son un `Set`, por lo que es normal que queramos que un usuario tenga múltiples roles (`ROLE_USER`, `ROLE_ADMIN`, `ROLE_TEAM_OWNER`, etc.) y de esa forma al definir las reglas no tener que asumir herencias entre roles, ni tener que redefinir todas las reglas si decidimos agregar un rol.

Con esto así, creamos nuestra página de login. Agregamos a nuestro controller:

```

@RequestMapping("/login")
public ModelAndView login() {
    return new ModelAndView("login");
}

```

Luego creamos `login.jsp` acorde:

```

<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core_rt"%>
<%@ taglib prefix="spring" uri="http://www.springframework.org/tags"%>

<html>
<body>
<c:url value="/login" var="loginUrl" />
<form action="${loginUrl}" method="post" enctype="application/x-www-form-urlencoded">
    <div>
        <label for="username">Username: </label>
        <input id="username" name="j_username" type="text"/>
    </div>
    <div>
        <label for="password">Password: </label>
        <input id="password" name="j_password" type="password"/>
    </div>
</div>

```

```

        <label><input name="j_rememberme" type="checkbox"/> <spring:message
code="remember_me"/></label>
    </div>
    <div>
        <input type="submit" value="Login!"/>
    </div>
</form>
</body>
</html>

```

Finalmente, y sólo para el ejemplo, agregamos nuestro `AdminController`

```

@RequestMapping("/admin")
@Controller
public class AdminController {

    @RequestMapping("/")
    public ModelAndView index() {
        return new ModelAndView("admin/index");
    }
}

```

Y la vista `admin/index.jsp`

```

<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core_rt"%>

<html>
<head>
    <link rel="stylesheet" href="<c:url value="/css/style.css"/>" />
</head>
<body>
<h2>This is the admin dashboard</h2>
</body>
</html>

```

Del mismo modo, agregamos nuestro `ErrorController` :

```

@Controller
public class ErrorController {

    @RequestMapping("/403")
    public ModelAndView forbidden() {
        return new ModelAndView("403");
    }
}

```

Y la vista `403.jsp` :

```

<%@ taglib prefix="c" uri="http://java.sun.com/jstl/core_rt"%>

<html>
<head>
    <link rel="stylesheet" href="<c:url value="/css/style.css"/>" />

```

```
</head>
<body>
<h1>You shall not pass!</h1>
</body>
</html>
```

Corremos la aplicación y vemos que la autenticación está funcionando como esperamos.

Logging

Logback es una librería de logging del mismo creador de Log4J, que, basado en los problemas y errores que tuvo en Log4J, decidió crear una nueva librería que por diseño pueda evitar estas dificultades. El resultado es Logback, la primer (y hasta donde conoco única) implementación nativa del standard SLF4J, que no depende de un wrapper para satisfacer su API.

Logback es sumamente simple de configurar. El primer paso es agregar todas las dependencias necesarias en el pom padre:

```
<org.slf4j.version>1.7.5</org.slf4j.version>
<logback.version>1.1.2</logback.version>
<logback-ext-spring.version>0.1.1</logback-ext-spring.version>
```

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>${org.slf4j.version}</version>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>${logback.version}</version>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-core</artifactId>
  <version>${logback.version}</version>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>jcl-over-slf4j</artifactId>
  <version>${org.slf4j.version}</version>
</dependency>
<dependency>
  <groupId>org.logback-extensions</groupId>
  <artifactId>logback-ext-spring</artifactId>
  <version>${logback-ext-spring.version}</version>
  <scope>runtime</scope>
</dependency>
```

Y en el módulo webapp

```

<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-core</artifactId>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>jcl-over-slf4j</artifactId>
</dependency>
<dependency>
  <groupId>org.logback-extensions</groupId>
  <artifactId>logback-ext-spring</artifactId>
</dependency>

```

Y lo único que resta es establecer la configuración a ser usada creando en `src/main/resource/` un archivo llamado `logback.xml`

```

<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property name="defaultPattern" value="%d{HH:mm:ss.SSS} [%thread] %-5level
%logger{36} - %msg%n" />

  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>${defaultPattern:-}</pattern>
    </encoder>
  </appender>

  <root level="DEBUG">
    <appender-ref ref="STDOUT" />
  </root>
</configuration>

```

Como vimos en clase, esto va a configurar un root logger (que recibe todos los mensajes de la aplicación) con un filtro a nivel `DEBUG`, enviando todo el output a un appender llamado `STDOUT` que no es otra cosa que la consola. Esto si bien puede ser útil en desarrollo, claramente no es óptimo para un ambiente productivo, donde no se debe loguear ni en consola ni en modo `DEBUG`. Ya veremos en la próxima clase como mejorar esta configuración.